AIRLINE DASHBOARD

CSU11013

PROGRAMMING PROJECT

TEAM 22

MEMBERS

1. FILIP ADAMCZAK
2. IRENE BIJOY
3. PARIDDHI JUS ROY
4. VHIDULA ABIRAMI PRAKASH
5. KARTHIKEYAN VINODKUMAR

Group 22 Member Details

Name	T-Number	Major Contributions
Filip Adamczak	25382444	Developed the `TableWidget` and `Checkbox` classes for loading and displaying flight data. Integrated the 'Show Diverted' and 'Show Cancelled' checkboxes and ensured visualisations update dynamically with the table. Worked on `Graphs` class and `Query` class. Managed code cleaning and performance improvements.
Irene Bijoy	25383531	Built the bar chart and search bar in `QueryScreen`, filtering flights by column, search string, and checkbox state. Added a cancelled flights and diverted flights table to the Passenger screen.
Pariddhi Jus Roy	25348641	Implemented table pagination using `nextPage()` including wrap-around behavior and handling for partial final pages via the modulo operator. And added labels and legend to the Map screen.
Vhidula Abirami Prakash	25342659	Maintained code documentation throughout development. Contributed to the search page graphs and implemented the interactive graph dropdowns.
Karthikeyan Vinodkumar	25341919	Developed `VisualsClass` and the overall user interface (UI). Worked on elements of `Graphs` class, `Screen` class and `PassengerScreen` class. Implemented `MapScreen` including airport dots that scale proportionally with flight traffic.

Abstract

The Airline Dashboard is an interactive data visualization application developed using Processing to analyze flight data. The application consists of three screens - Passenger, Query, and Map that display data using tables, filters, and graphs. Users can search and filter flights based on different parameters, with results updating in real time. Custom UI components such as dropdowns and checkboxes are used to improve interaction. Data is managed efficiently using filtering and caching techniques to maintain smooth performance. The project demonstrates the use of programming and visualization techniques to build a structured and responsive application.

Literature Review

The Processing documentation was the main resource, which provided guidance on core functions such as table loading, Bezier curve rendering, and event handling. For more specific implementation questions, the Processing community forum and Stack Overflow were useful references. General programming references were also consulted when dealing with Java-related questions that arose during development. The flight dataset itself was sourced from the US Bureau of Transportation Statistics. No external Processing libraries were used at any point. Every UI component, including dropdowns, checkboxes, and buttons, was built from scratch using the core Processing API.

Project Details

The Airline Dashboard is an interactive data visualisation application built in Processing, to explore a dataset of commercial flights. It reads flight records from a CSV file and displays the data across three screens, navigated through a top navbar.

The application is structured around a base Screen class, with three screens — PassengerScreen, QueryScreen, and MapScreen — each extending it. The main sketch calls draw() and mousePressed() on whichever screen is active. A Visuals class holds all colours, fonts, and layout constants in one place so the design stays consistent across the whole app.

Flight data is loaded using Processing's built-in loadTable() and managed by a TableWidget class that handles rendering and pagination. QueryScreen lets users filter flights across any of the columns, with results instantly reflected in three live charts — pie, bar, and line — handled by the Graphs class. The MapScreen draws flight routes on a US map, with airport dots sized by traffic volume. A caching system ensures only the relevant date's data is processed.

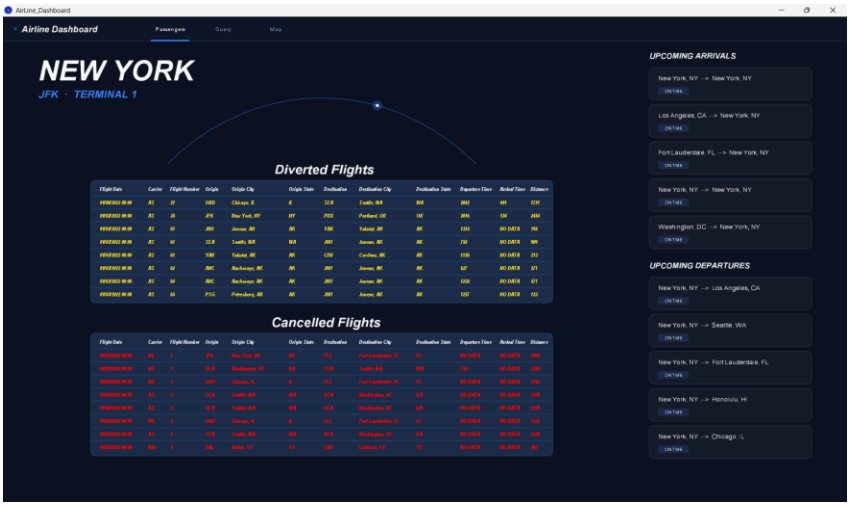
Execution and Results

Architecture and Implementation

1. Component Structure
- The code is split into .pde files, with each UI element implemented as a self-contained class that handles its own rendering. This meant that when something broke, it was always confined to one file. The Graphs class is the most complex, consisting of three chart types with five Dropdown instances for column selection. The Query class handles keyboard input and search box rendering. Each screen class has these elements with its own draw() and mousePressed() calls.
2. Data Pipeline
- Flight data is loaded once in setup() through the loadTable() function. The TableWidget class uses this data and converts the Cancelled and Diverted columns from "0"/"1" to "YES"/"NO" for readability. In QueryScreen, filterFlights() only rebuilds the table when the query changes. This prevents scanning the full dataset in every frame. With every rebuild, the Graphs object receives an updateData() call, keeping all three graphs updated with the current filtered results.
3. Font & Asset Management
- All fonts and images are loaded once in setup() and stored as global PFont and PImage constants, ensuring a consistent style.

Screen Results

1. Passengers Screen
- The default screen displays a departure board with an animated flight path, an arrivals and departures panel, and two tables showing all diverted and cancelled flights. The page animates a Bezier curve using Processing's bezierPoint() function.

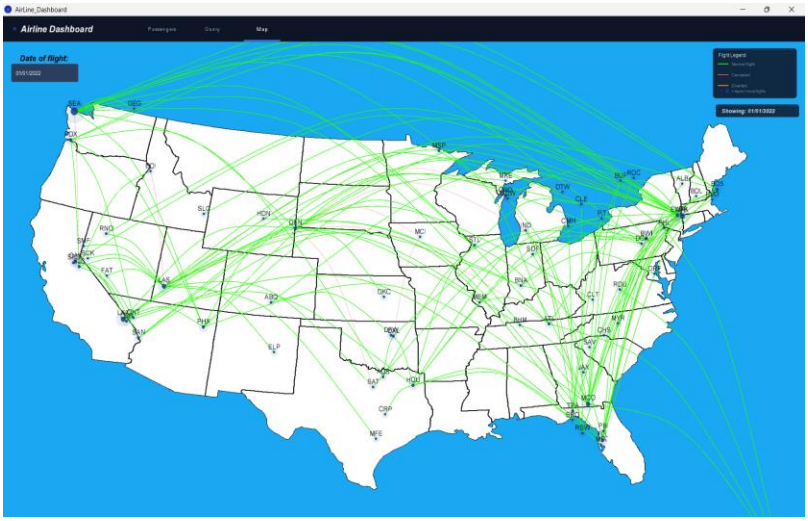


2. Query Screen

– The Query screen has a search box, a column-selection dropdown, and two checkboxes — Show Diverted and Show Cancelled — to filter the dataset in real time. Three charts update automatically to reflect the matching rows. When no rows match the query, a "No records match" message appears on both table and the chart area. Pagination buttons allow navigation through the filtered results.

3. Map Screen

– The Map screen renders US flight routes on a background map image. Each flight for the selected date is drawn from origin to destination, color-coded by status — green for normal, red for cancelled, and amber for diverted. Airport coordinate points resize with the number of flights at that location. A date-selection dropdown filters routes by Flight Date. Selecting a new date triggers a cache rebuild and the curves update.



Conclusion

It is evident that our project contains all the basic features that would be expected from an airline dashboard, and all these various elements of the program are implemented in a concise and efficient manner. The three screens have their own individual classes which makes editing the code for each of them simple. Also, features like dropdowns, buttons and checkboxes, also have their own classes which maintained their uniformity throughout the program. And lastly, in terms of the visuals, the font size, font color, the various background colors were all stored in a class in a single .pde file which made accessing them easier and helped maintain the look of our program.

References

1. *Reference*. (n.d.). Processing. https://processing.org/reference

2. *Processing Forum*. (2018). Processing.org. https://forum.processing.org/

3. Stack Overflow. (2024). *Stack Overflow - Where Developers Learn, Share, & Build Careers*. Stack Overflow. https://stackoverflow.com/